

Assignment 2

Factory Method and Abstract Factory

Student: Tulkibay Miras
Group: SE-2528
Course: Software Design Patterns
Repository: github.com/mirast03/asiksdp

1. Introduction

This project is a small Java logistics application that demonstrates two creational design patterns in one program. Factory Method is used to select the delivery transport (Truck for ROAD and Ship for SEA). Abstract Factory is used to create a consistent family of UI components for WINDOWS or MACOS. The two choices are independent, so any supported delivery mode can be combined with any supported UI platform.

The application is console-based. At runtime the user selects a delivery mode and a UI platform. The startup code chooses the correct Logistics creator and GUIFactory, while the main application works through abstractions rather than concrete classes.

2. Factory Method

Transport is the product abstraction. Truck and Ship are concrete products. Logistics is the creator abstraction and defines the shared planDelivery operation. RoadLogistics and SeaLogistics override createTransport(), which is the actual Factory Method.

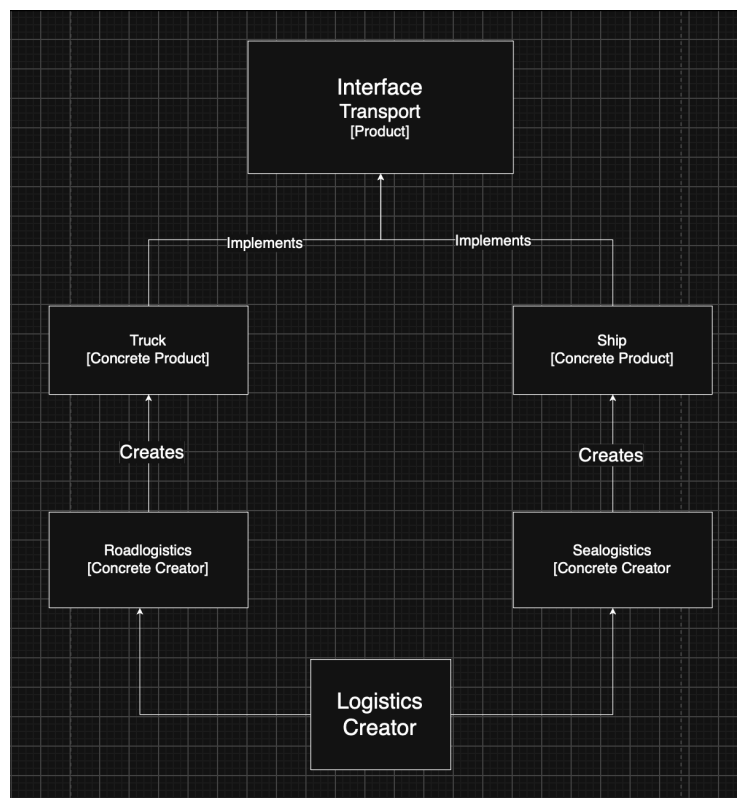


Figure 1. Factory Method structure used by the logistics module.

3. Abstract Factory

GUIFactory defines creation methods for Button and Checkbox. WindowsFactory creates WindowsButton and WindowsCheckbox, while MacOSFactory creates MacOSButton and MacOSCheckbox. DeliveryApplication receives GUIFactory and Logistics through its constructor and uses the created components only through their interfaces.

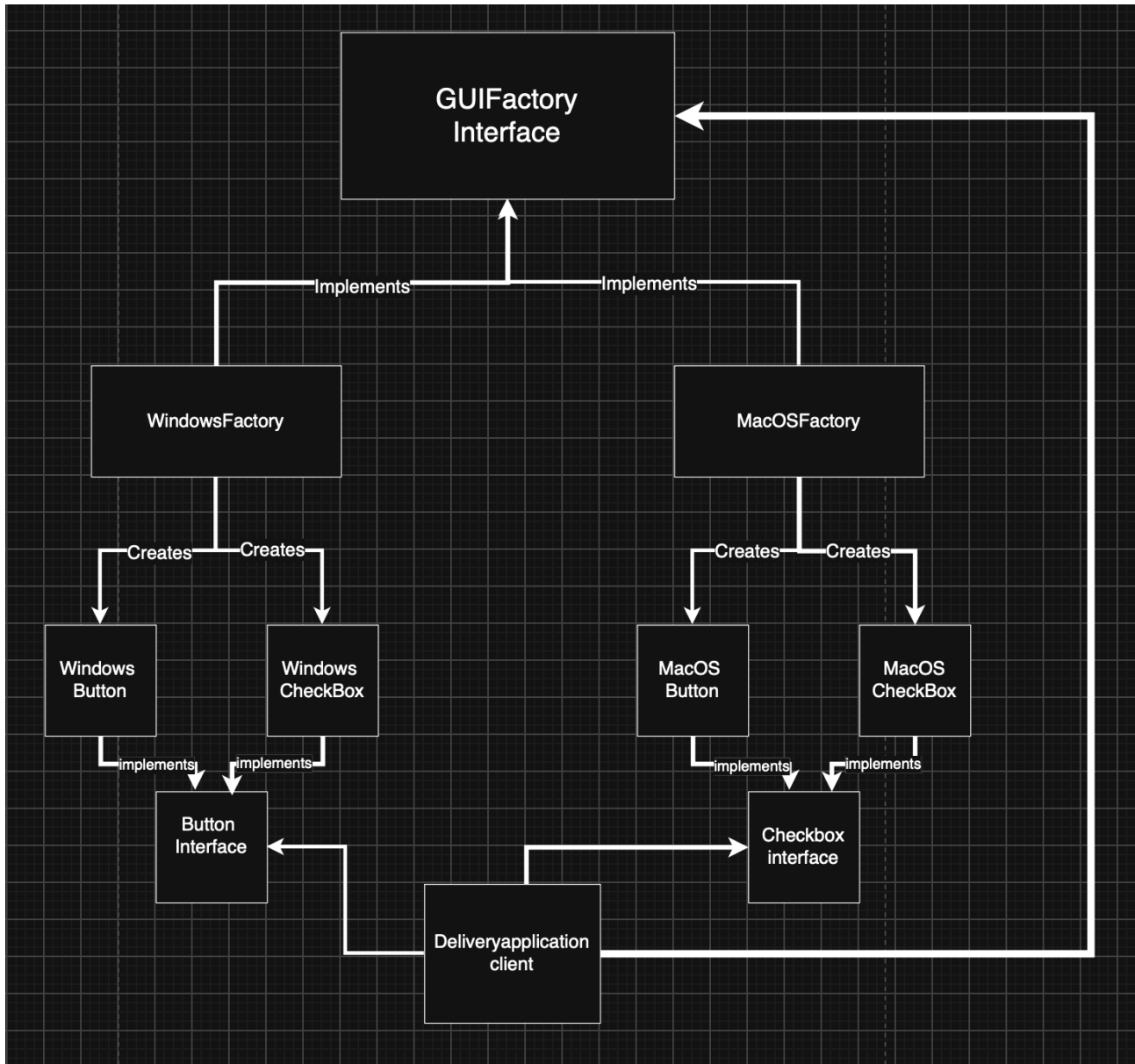


Figure 2. Abstract Factory structure for Windows and macOS UI families.

Runtime composition

The delivery choice (ROAD/SEA) and platform choice (WINDOWS/MACOS) are selected separately. This allows ROAD+WINDOWS, ROAD+MACOS, SEA+WINDOWS, and SEA+MACOS without changing the client logic.

4. Clean Code Evidence

The following examples are taken from the implementation and show practices used to keep the design understandable and easy to extend.

1. Meaningful names

```
public interface Transport {  
    void deliver(String cargo, String destination);  
}
```

Benefit: Transport, deliver, cargo, and destination describe the domain directly. This reduces the need for explanatory comments.

2. Small methods

```
public Transport createTransport() {  
    return new Truck();  
}
```

Benefit: The factory method has one clear responsibility: create the correct transport object.

3. Avoiding duplication

```
public void planDelivery(String cargo, String destination) {  
    Transport transport = createTransport();  
    transport.deliver(cargo, destination);  
}
```

Benefit: Shared delivery workflow is kept in Logistics instead of duplicating the same workflow in RoadLogistics and SeaLogistics.

4. Data abstraction (Chapter 6)

```
Transport transport = createTransport();  
transport.deliver(cargo, destination);
```

Benefit: Logistics depends on the Transport abstraction. It does not need to know whether the object is a Truck or a Ship.

5. Objects and encapsulation (Chapter 6)

```
public DeliveryApplication(GUIFactory factory, Logistics logistics) {  
    button = factory.createButton();  
    checkbox = factory.createCheckbox();  
    this.logistics = logistics;  
}
```

Benefit: DeliveryApplication receives collaborators through abstractions and keeps them as internal object state. Concrete creation details stay outside the client.

5. Design Reflection

Adding a new transport: create a new Transport implementation and a new Logistics subclass whose createTransport() returns that product. Existing delivery workflow can stay unchanged.

Adding a new UI family: create a new GUIFactory implementation and matching Button and Checkbox classes. DeliveryApplication does not need to change because it depends on interfaces.

Adding a new UI product such as TextField: extend GUIFactory with createTextField(), add a TextField interface, and implement a matching TextField for every UI family. Existing factories must then be updated because Abstract Factory keeps product families consistent.

6. Verification

The program was exercised with all required supported combinations and invalid selections. The table records the expected behavior and test result. Representative console evidence is shown below and on the next page.

Input	Expected / observed behavior	Result
ROAD + WINDOWS	Windows UI; Truck delivery	PASS
SEA + WINDOWS	Windows UI; Ship delivery	PASS
ROAD + MACOS	macOS UI; Truck delivery	PASS
SEA + MACOS	macOS UI; Ship delivery	PASS
PLANE + valid UI	Clear unsupported delivery message; clean stop	PASS
SEA + LINUX	Clear unsupported UI message; clean stop	PASS
Missing/empty input	Rejected by validation; program does not continue with null/default object	PASS

```

C:\Users\user> java -jar C:\Users\user\Downloads\SoftwareDesignPatterns.jar
Enter delivery mode:
> ROAD
Enter UI platform (WINDOWS or MACOS):
> WINDOWS
Delivery mode: ROAD
UI platform: WINDOWS
Rendering window button
Rendering message button
Truck delivery laboratory equipment database warehouse
Process finished with exit code 0
  
```

ROAD + WINDOWS

```

C:\Users\user> java -jar C:\Users\user\Downloads\SoftwareDesignPatterns.jar
Enter delivery mode:
> ROAD
Enter UI platform (WINDOWS or MACOS):
> MACOS
Delivery mode: ROAD
UI platform: MACOS
Rendering window button
Rendering message button
Truck delivery laboratory equipment database warehouse
Process finished with exit code 0
  
```

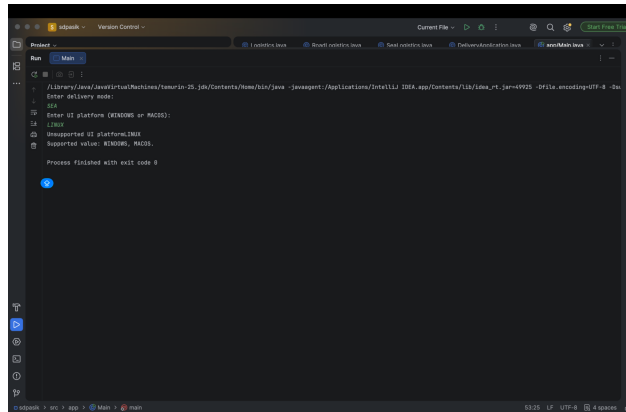
ROAD + MACOS

```

C:\Users\user> java -jar C:\Users\user\Downloads\SoftwareDesignPatterns.jar
Enter delivery mode:
> PLANE
Enter UI platform (WINDOWS or MACOS):
> WINDOWS
Unsupported delivery mode: PLANE
Unsupported window: PLANE
Process finished with exit code 0
  
```

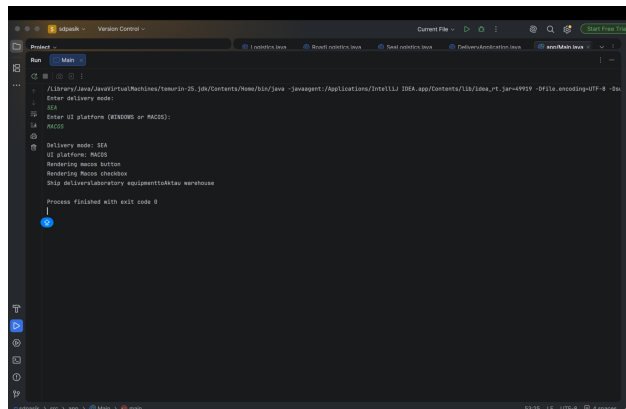
Unsupported delivery mode (PLANE)

Verification Evidence (continued)



```
Product -> Main
Run
Main
Enter delivery mode: SEA
Enter UI platform (WINDOWS or MACOS): LINUX
Unsupported UI platform: LINUX
Reported value: WINDOWS, MACOS.
Process finished with exit code 0
```

Unsupported UI platform (LINUX)



```
Product -> Main
Run
Main
Enter delivery mode: SEA
Enter UI platform (WINDOWS or MACOS): MACOS
Delivery mode: SEA
UI platform: MACOS
Rendering menu button
Rendering menu checkboxes
This deliveryLaboratory equipmentfabrika warehouse
Process finished with exit code 0
```

SEA + MACOS

7. Conclusion

The project demonstrates Factory Method and Abstract Factory in one Java application. Factory Method isolates transport creation behind Logistics subclasses, while Abstract Factory keeps Windows and macOS UI components in consistent families. The client code depends on interfaces, making the design easier to extend without rewriting the main delivery workflow.

8. References

1. Refactoring.Guru - Factory Method pattern.
2. Refactoring.Guru - Abstract Factory pattern.
3. Robert C. Martin, *Clean Code*, especially Chapter 6: Objects and Data Structures.
4. Course assignment specification: Assignment 2 - Factory Method and Abstract Factory.

Submission note: The assignment specification requires JDK 17. The supplied IntelliJ screenshots show a Temurin 25 runtime, so the final repository should be re-run with JDK 17 before submission and defense.